

Programmation en langage C – EI3

Travaux Pratiques

Objectif(s)

★ pointeurs, structures, allocation dynamique

★ modélisation d'un arbre binaire

L'objectif de ce TP est double. Il s'agit de modéliser un réseau de métro (on prendra exemple sur celui de Paris) et ensuite de pouvoir calculer le meilleur trajet entre deux stations.

Avant de commencer une question, il est important de bien lire TOUTE la question en entier.

Ce (très) long TP demande de maîtriser allocation dynamique de mémoire, pointeurs et structures. Posez-vous la question, quand vous manipulez un pointeur, s'il pointe bien vers quelque chose qui existe (alloué dynamiquement ou statiquement).

1 Modèle du réseau de métro

Avant de commencer, on va étudier un modèle simplifié d'un petit réseau (figure 1) à 2 lignes (rouge et bleue) et 10 stations (de A à J). On peut voir que les stations sont reliées par des sections : les sections vont d'une station à une autre, et sauf exceptions, il y a deux sections reliant deux stations, une dans chaque sens. Ces exceptions existent aussi pour le réseau parisien, sur la ligne 10 et sur la ligne 7bis, d'où la nécessité de les gérer et d'avoir des sections qui relient deux stations *que* dans un sens.

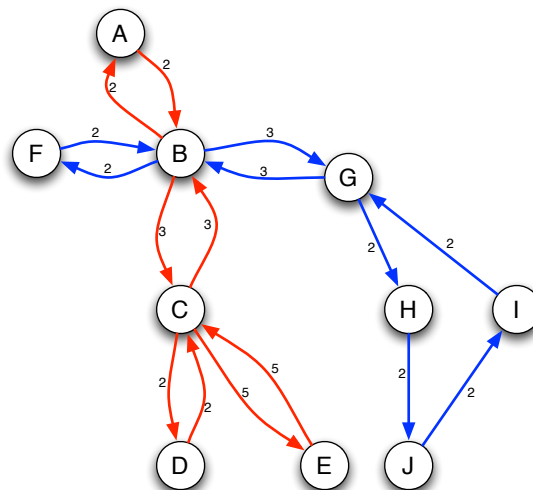


Figure 1: Petit exemple de réseau

De plus, chaque section possède une valeur appelée *durée* qui est le temps mis (en minutes) pour aller d'une station à une autre.

On cherchera donc, dans ce TP, à modéliser un réseau de ce type et à calculer le chemin le plus court pour aller d'une station à une autre (par exemple, pour aller de la station A à la station I, il faut aller de A à B, changer à B, puis aller à G, H, J et enfin I). Ce petit exemple est trop simple pour qu'il y ait plusieurs chemins possibles pour aller d'une station à une autre, mais c'est bien sûr le cas dans la vraie vie.

2 Génération du réseau

2.1 Les stations

Question 1 – Structures

1. Une station sera composée d'un nom (alloué dynamiquement), d'une latitude et d'une longitude. Dans un fichier `metro.h`, écrire une structure `t_station` correspondante.
2. Un métro est composé d'un tableau de stations (alloué dynamiquement) et du nombre de stations qu'il contient. Dans ce même fichier, écrire une structure `t_metro` correspondante.

Question 2 – Stations définies dans le fichier

On dispose d'un fichier texte (un pour l'exemple, appelé `ex-stations.txt` et un pour le réseau de métro parisien appelé `paris-stations.txt`). Ces fichiers sont constitués :

- du nombre de station (suivi d'un saut à la ligne),
- puis, pour chacune des stations, le nom de la station, sa latitude et sa longitude séparés par le caractère : (deux-points). Il y a ensuite un saut à la ligne pour chaque station.

Par le petit exemple, nous avons donc

```
10
A:2.125:0.5
B:2.75:1.75
C:2.75:3.375
D:2.75:5
E:4.25:5
F:1.25:1.75
G:4.5:2
H:5.125:3.375
I:6.25:3.375
J:5.125:5
```

1. Dans `metro.c`, écrire une fonction `readMetro` qui ouvre un fichier dont le nom est passé en paramètre et remplit un `t_metro` (on se demandera s'il faut un pointeur ou pas). Après avoir alloué le tableau de station, on appellera successivement la fonction `readStation` de la question suivante pour lire chaque station,. Après avoir lu l'entier qui indique le nombre de station, pensez bien à lire le caractère de saut de ligne `"\n"`.
2. Dans ce même fichier `metro.c`, écrire la fonction `readStation` qui lit une station depuis un fichier ouvert (`FILE*`) et qui remplit la `t_station` passée en paramètre.
On notera que pour lire un nom avec les espaces jusqu'au : (deux-points), on utilisera la chaîne `"%[^:]"` au lieu du classique `"%s"`. De même, pour la lecture des deux flottants suivants, on utilisera `":%f:%f\n"` pour lire les : (deux-points), la latitude, : (deux-points), la longitude et le caractère de fin de ligne¹. Il faudra aussi faire l'allocation de mémoire pour le nom de la station avant la recopie.
3. Enfin, dans un fichier `main.c`, écrire une fonction `main` qui lit le fichier `paris-stations.txt` et qui affiche le nom et la position de la station n° 100.

Question 3 – Recherche d'une station

1. Dans `metro.c`, écrire une fonction `findStation` qui prend un métro et une chaîne de caractères et renvoie un pointeur vers la station qui a ce nom si la station existe, ou `NULL` sinon.
Pour effectuer la comparaison entre deux chaînes de caractères (deux tableaux de `char`), on utilisera la fonction `strcmp` définie dans `string.h`.
2. En utilisant cette fonction, afficher dans le `main`, la position (latitude, longitude) de la station `"Jussieu"`.

¹Il s'agit en fait du `scanf` utilisé avec une expression régulière. Plus d'information par exemple ici : https://www.gnu.org/software/libc/manual/html_node/String-Input-Conversions.html

2.2 Les lignes

Question 4 – Structures

1. Une ligne est composée d'un nom (alloué dynamiquement), d'une couleur (chaîne de 10 caractères) et d'un temps moyen entre deux trains (l'attente, égal à la demie-période entre deux trains sur la ligne). Dans le fichier `metro.h`, rajouter le code définissant le type `t_line`.
2. Complétez la structure `t_metro` pour qu'elle contienne aussi un tableau de lignes (alloué dynamiquement) et le nombre de lignes qu'il contient.

Question 5 – Lignes définies dans le fichier

On dispose maintenant des fichiers `ex-lignes.txt` et `paris-lignes.txt` qui contiennent la liste des stations (comme précédemment) suivie de la description des lignes. En plus du nombre de station et la description de chaque station, le fichier comporte maintenant :

- le nombre de lignes,
- puis, pour chacune des lignes, le nom de la ligne, sa couleur² et l'attente, le tout séparé par : (deux-points) et terminé par un saut de ligne.

Pour le petit réseau d'exemple, nous avons donc, après la définition des stations :

```
2
Ligne A:#FF0000:4
Ligne B:#0000FF:5
```

1. Compléter la fonction `readMetro` pour qu'elle lise les lignes du métro en s'appuyant sur la fonction `readLine` de la question suivante.
2. Dans `metro.c`, écrire la fonction `readLine` qui lit une ligne depuis un fichier ouvert (`FILE*`) et qui remplit la `t_line` passée en paramètre.
De même que pour la question 2.1, on utilisera la chaîne `"%[^:]"` au lieu du classique `"%s"` et on n'oubliera pas de lire les : (deux-points) ainsi que le caractère de fin de ligne. On utilisera aussi l'allocation dynamique (et la recopie) pour le nom de la ligne.
3. Tester les fonctions en affichant dans la fonction `main` le nom et la couleur de la 10^{ème} ligne du fichier `paris-lignes.txt`.

2.3 Les sections

Question 6 – Structures

1. Une section est composée d'une référence vers une station de début (pointeur), une référence vers une station de fin (pointeur), une référence vers une ligne, une durée (en minutes) et une direction³. Écrire la structure `t_section` correspondante.
2. On cherche à insérer dans la structure `t_station` un tableau de sections pour mémoriser toutes les sections qui **partent** de cette station. Rajouter dans la structure `t_station` un tableau de 10 `t_section*` (alloué statiquement⁴) ainsi que le nombre de sections.
3. Modifier la fonction `lireStation` pour mettre à zéro le nombre de sections par stations.

²quelques caractères définissant la couleur en code HTML, c'est-à-dire le caractère #suivi en hexadécimal des valeurs de Rouge, Vert et Bleu. Par exemple `#FFFF00` pour le jaune.

³La direction sera une valeur arbitraire 0 ou 1 qui permettra de distinguer deux sections de la même ligne mais qui ne vont pas dans le même sens. Par exemple, les sections `A→B`, `B→C` et `C→D` de la ligne rouge auront 1 comme direction, alors que les sections `B→A`, `C→B` et `D→C` auront 0 comme direction.

⁴N'ayant pas accès au nombre de section dans les fichiers, on considèrera qu'il n'y a pas plus que 10 sections dans une stations. On aura donc un tableau statique de 10 pointeurs vers des sections, et on allouera dynamiquement un `t_section` à chaque pointeur quand une section se présentera, voir plus loin la fonction `readSection`.

Question 7 – Sections définies dans le fichier

On dispose maintenant des fichiers `ex.txt` et `paris.txt` qui contient la liste des stations (comme précédemment), suivie de la description des lignes. Mais ici la description des lignes est complétée par la description des sections de chaque ligne.

Ainsi, chaque ligne est composée de :

- comme précédemment, du nom de la ligne, de sa couleur et de l'attente, le tout séparé par : (deux-points) et terminé par un saut de ligne,
- puis le nombre de sections dans la direction 0,
- pour chacune de ses sections, le nom de la station de départ, le nom de la station d'arrivée et la durée du trajet, le tout séparé par : (deux-points) et terminé par un saut de ligne,
- le nombre de sections dans la direction 1,
- et enfin pour chacune de ses sections, le nom de la station de départ, le nom de la station d'arrivée et la durée du trajet (exactement comme pour la direction 0).

Ainsi, pour notre petit exemple, la partie du fichier décrivant les lignes est donnée par

```
2
Ligne A:#FF0000:4
4
A:B:2
B:C:3
C:D:2
C:E:5
4
B:A:2
C:B:2
D:C:2
E:C:5
Ligne B:#0000FF:5
4
F:B:2
B:G:3
G:H:2
H:J:2
4
J:I:2
I:G:2
G:B:3
B:F:2
```

1. Modifier la fonction `readLine` pour qu'elle lise la description des sections, en s'appuyant sur la fonction `readSection` de la question suivante. Si nécessaire, la modifier aussi pour qu'elle accepte en paramètre un metro; modifier son appel.
2. Écrire une fonction `readSection` qui accepte en paramètre une ligne, un fichier ouvert, un metro et une direction. Cette fonction doit lire les informations d'une section, récupérer les stations de départ et d'arrivée (grâce à `findStation`) et rajouter ces informations dans le tableau de sections de la station de départ (ce tableau étant un tableau de pointeur vers des sections, il faut donc ici allouer une nouvelle section à rajouter dans le tableau de pointeur de sections de la station de départ; il ne faut pas oublier d'incrémenter le nombre de sections de la station de départ).
On veillera à afficher un message d'erreur si une des stations indiquées dans la section n'existe pas.
3. Dans le `main`, afficher toutes les sections qui partent de la station "Jussieu", sous la forme "Ligne_10: Jussieu->Gare_d'Austerlitz_(3min)" (il doit y en avoir 4 en tout).

Question 8

Il faut maintenant penser à proposer une fonction pour désallouer tout ce qui a été alloué.

- Écrire une fonction (dans `metro.c`) pour désallouer une station (donc désallouer toutes ses sections) ;
- Écrire une fonction pour désallouer une ligne ;
- Écrire une fonction pour désallouer le métro (donc toutes les lignes et toutes les stations).

3 Parcours du réseau



Pour parcourir le réseau et rechercher le plus court (en temps) chemin entre deux stations, nous allons utiliser l'algorithme de Dijkstra⁵ (du nom de l'informaticien Edsger Dijkstra⁶ qui l'a publié en 1959). Celui-ci se base sur la notion de graphe et de parcours de graphe. C'est exactement ce que nous avons mis en place avec nos stations (qui sont les nœuds du graphe) et nos sections (les arrêtes qui relient les nœuds du graphe) qui relient les stations.

3.1 Recherche du plus court chemin

Question 9 – Structures

Pour pouvoir appliquer l'algorithme de Dijkstra, il nous encore modifier légèrement nos structures.

1. On a besoin aussi pour les stations :

- d'un booléen indiquant si la station a été validée ou non par l'algorithme (on considèrera qu'une station a été *validée* par l'algorithme quand il a trouvé le plus court chemin pour y accéder),
- d'un entier indiquant la distance (en minute) entre la station de départ et cette station,
- ainsi qu'une référence vers une section qui fait arriver à cette station pour le plus court chemin (on parlera par la suite de *chemin*)

Modifier en conséquence la structure `t_station`.

Question 10 – Algorithme de Dijkstra

1. Dans un fichier `Dijkstra.c`, écrire une fonction `Dijkstra` qui prend en paramètres d'entrée un métro, une station de départ et une station d'arrivée (toujours à vous de voir si c'est un pointeur ou pas). Cette fonction doit mettre en œuvre l'algorithme de Dijkstra énoncé ci-dessous et s'appuie sur les fonctions `chercheMinimum` et `majDistance` des questions suivantes :

```

Entrées : station de départ, station de début
// Initialisation
1 ...
// Parcours
2 Station courante ← station de départ
3 tant que Station courante ≠ station d'arrivée faire
4   station courante ← chercheMinimum()
5   station courante est validée
6   pour sec parmi toutes les sections de la station courante faire
7     majDistance(sec)
8   fin
9 fin

```

Pour l'initialisation, il s'agit, pour toutes les stations, de les indiquer comme *non validées* (non atteinte), de leur mettre leur distance à ∞ (en pratique, on prendra une constante entière très grande par rapport aux temps de trajets dans un réseau) et le pointeur vers le chemin à `NULL`. Il faut par contre, mettre la distance de la station de départ à 0 (il faut un temps nul pour aller de la station de départ à la station de départ!).

⁵http://fr.wikipedia.org/wiki/Algorithme_de_Dijkstra

⁶http://fr.wikipedia.org/wiki/Edsger_Dijkstra

2. Dans `Dijkstra.c`, écrire la fonction `chercheMinimum` qui prend en paramètres d'entrée un métro et qui renvoie un pointeur vers la station qui a la plus petite distance de toutes les stations non encore validées. Pour cela, il faut procéder en deux étapes
 - (a) trouver *une* station non encore validée. Elle servira à obtenir une valeur initiale pour chercher le minimum;
 - (b) ensuite, parcourir toutes les stations non encore validées et trouver celle qui donne la distance la plus petite.
3. Toujours dans `Dijkstra.c`, écrire la fonction `majDistance` qui effectue la mise-à-jour des distances. Cette fonction prend une section en entrée, et modifie la distance de la station de fin de cette section selon l'algorithme :

Entrées : section

Données : `sDeb` : la station de début de la section

Données : `sFin` : la station de fin de la section

```

1 si distance de sFin > distance de sDeb + durée de la section alors
2   distance de sFin ← distance de sDeb + durée de la section
3   chemin de sFin ← section
4 fin
```

4. Dans le `main`, afficher à quelle distance (en minute) se trouve la station "Avron" de "Jussieu". Tester avec d'autres stations (sur la même ligne par exemple).

3.2 Affichage du plus court chemin

Maintenant que l'algorithme de Dijkstra permet de connaître les distances minimales entre la station de départ et celle d'arrivée, il faut maintenant afficher le chemin le plus court.

Le principe est le suivant : on commence par la station d'arrivée, et on remonte vers la station de départ en suivant les chemins de chaque station (ce sont les sections que l'on a mémorisé à chaque fois que l'on faisait une mise-à-jour des distances et que la distance en passant par la section considérée était plus avantageuse que la distance sans).

Question 11

1. Dans `Dijkstra.c`, écrire une fonction `afficheChemin` qui prend en paramètre un métro, une station de départ et une station d'arrivée, et qui affiche le chemin. On se contentera d'afficher le chemin à l'envers (de la station d'arrivée à la station de départ) en indiquant chaque section et le temps à chaque station.
Exemple, pour aller de "Pont_Marie" à "Invalides", on affichera

```

Ligne 8 : De Concorde à Invalides (9 min)
Ligne 1 : De Tuileries à Concorde (7 min)
Ligne 1 : De Palais Royal - Musée du Louvre à Tuileries (6 min)
Ligne 7 : De Pont Neuf à Palais Royal - Musée du Louvre (5 min)
Ligne 7 : De Châtelet à Pont Neuf (3 min)
Ligne 7 : De Pont Marie à Châtelet (2 min)
```

3.3 Bonus - Amélioration du modèle

Ce modèle a quelques limitations. Par exemple, les temps de changements à une station, et les temps d'attente d'un métro ne sont pas pris en compte. Pourtant, il est facile de les estimer :

- On dispose déjà de la période entre deux trains sur une même ligne. On peut donc en déduire un temps moyen d'attente sur une ligne.
 - De plus, on peut supposer/estimer que les temps de changements de ligne à une station dépendent de la taille de la station. Par exemple, en 1^{ère} estimation, on peut dire qu'une station qui a jusqu'à 2 sections demande 2 minutes de changements, une station jusqu'à 4 ou 6 sections demande 5 minutes et qu'une station avec plus de sections (genre Montparnasse ou Les Halles) demande 8 ou 10 minutes.
1. Écrire (dans `Dijkstra.c`) une fonction qui estime le temps de changement d'une station donnée en paramètre.

-
2. Modifier la fonction `majDistance` pour modifier le temps calculé pour effectuer la section si, entre le chemin qui amène à la station de début de la section et la section, il y a un changement de ligne. Prendre en compte le fait que le chemin qui amène à la station de début de la section peut ne pas exister (égal à `NULL`), comme quand on considère les sections qui partent de la station de départ. Dans ce cas, il n'y a pas de temps de changement, mais juste le temps moyen d'attente sur la ligne.
 3. On peut aussi faire la distinction entre temps de trajet et temps d'attente ou de marche (temps de changement). Faire les modifications nécessaires pour obtenir l'affichage du chemin suivant (ici de Jussieu à Trocadéro) :

```
*Ligne 10
De Jussieu à La Motte-Picquet - Grenelle
  (18 min, dont 5 à pied ou attente)
*Ligne 6
De La Motte-Picquet - Grenelle à Trocadéro
  (28 min, dont 6 à pied ou attente)
```

4. Enfin, je tiens à la disposition de qui le veut la liste des stations de RER, les différentes sections, ainsi que les correspondances à pied (Châtelet et Les halles ou encore Auber et Opéra). Seuls les temps de trajets ne sont pas saisis, ainsi que les temps d'attente... Les fichiers datent d'une dizaine d'année, donc certaines nouvelles stations n'y figurent pas, il faudrait les mettre à jour.